LiveData vs Flow

LiveData: Específico para Views no Android, respeita o ciclo de vida (onPause/onStop) e é ideal para dados vinculados à UI.

LiveData.kt

```kotlin
private val _userLiveData = MutableLiveData<User>()
val userLiveData: LiveData<User> get() = _userLiveData
```

LiveData.kt

```kotlin
viewModel.userLiveData.observe(viewLifecycleOwner) { user →
    textView.text = user.name
}
```

Flow: Flexível e poderoso, funciona fora do ciclo de vida do Android, sendo ideal para streams de dados mais complexos.

Flow.kt

```kotlin
class UserRepository {
    fun getUserFlow(): Flow<User> = flow {
        emit(User(name = "John Doe", age = 30))
    }
}

class TestUserViewModel(repository: UserRepository) : ViewModel() {
    val userFlow: Flow<User> = repository.getUserFlow()
}
```

Flow.kt

```kotlin
lifecycleScope.launch {
    viewModel.userFlow.collect { user →
        textView.text = user.name
    }
}
```

Streams de Dados:

LiveData: Melhor para dados únicos ou vinculados diretamente à UI.

Flow: Ideal para streams contínuos de dados, como atualizações em tempo real.

Cancelamento Automático:

LiveData: Cuida automaticamente do ciclo de vida da UI, parando de observar quando a Activity ou Fragment está inativa.

Flow: Requer uso de lifecycleScope ou viewModelScope para garantir que as coletas sejam canceladas corretamente, evitando leaks de memória.

Interoperabilidade:

LiveData: Projetado exclusivamente para o Android, portanto, só funciona no ambiente Android.

Flow: Universal, pode ser usado em qualquer ambiente que suporte Kotlin, tornando-o mais flexível.